

JavaScript + TypeScript

Interview Guide

Full-Stack (Frontend & Node.js) — Basic to Advanced

September 2026

How to use this guide

This is an **interview handbook**, not a tutorial: every question is answered the way you'd say it out loud, with the specific terms an interviewer listens for bolded, the traps candidates fall into named explicitly, and the follow-up an interviewer typically asks next. If you have a week, work top to bottom. If you have a day, read the **Common Gotchas & Traps** section (Section 13) and the **Debugging Playbook** (Section 14) first, then skim the Question Index for anything unfamiliar, then the **Cheat Sheet** (Section 15) the morning of. This guide was built to serve **Full-Stack** candidates equally: browser runtime questions sit alongside Node.js-specific mechanics and TypeScript throughout, rather than in separate tracks.

Contents

1	Research Notes: Queries and Sources	iii
2	Interview Question Index	v
3	Core Language Fundamentals	1
4	Closures & Scope	4
5	this, call/apply/bind & Prototypes	6
6	The Event Loop (Browser)	9
7	Node.js Event Loop Specifics	12
8	Promises & Async/Await	14
9	Modules: CommonJS vs. ES Modules	17
10	Practical Coding Questions	19
11	TypeScript Fundamentals	22
12	TypeScript Advanced	25

13 Common Gotchas & Traps	29
14 Debugging Playbook	30
15 Cheat Sheet / Rapid Revision	33

1 Research Notes: Queries and Sources

Web search was available in this environment and was used, per instructions, before any outline was drafted. The category list and question selection below were shaped by what these searches actually returned, not by a pre-made syllabus. Where this guide calls something “frequently asked,” it appeared across three or more of the sources below.

Search queries used

- *most common JavaScript TypeScript interview questions 2026 frontend backend*
- *site:geeksforgeeks.org javascript interview questions closures event loop*
- *Glassdoor frontend interview javascript questions candidate experience*
- *JavaScript event loop microtask macrotask interview question output prediction*
- *TypeScript interview questions generics narrowing structural typing handbook*
- *this keyword prototype inheritance interview questions javascript common mistakes*
- *Node.js event loop phases timers vs process.nextTick microtask interview*
- *javascript debugging interview memory leak closures async bug scenario*
- *TypeScript utility types Partial Pick Omit Record interview coding questions*
- *javascript type coercion == vs === NaN hoisting output prediction interview gotchas*
- *javascript coding interview implement debounce throttle deep clone curry polyfill*
- *CommonJS vs ES modules interview question node.js require import*
- *promises async await error handling interview questions common mistakes unhandled rejection*
- *TypeScript interfaces vs types discriminated union enums interview questions*

Sources consulted

- **Candidate-experience / prep aggregators:** Glassdoor interview-experience pages for Front End Developer/Engineer roles at LinkedIn, Meta, Amazon, and CHECK24 ([glassdoor.com/Interview/...](https://www.glassdoor.com/Interview/)); GeeksforGeeks JavaScript and TypeScript interview tracks ([geeksforgeeks.org/javascript/...](https://www.geeksforgeeks.org/javascript/), [geeksforgeeks.org/typescript/...](https://www.geeksforgeeks.org/typescript/)); The Front End Interview Handbook (frontendinterviewhandbook.com) and GreatFrontEnd’s question bank and blog (greatfrontend.com), including its open-source `top-javascript-interview-questions` GitHub repo.
- **Authoritative references:** MDN Web Docs for closures, prototypal inheritance, `this`, Promises, and the event loop (developer.mozilla.org); the TypeScript Handbook for structural typing, generics, narrowing, and utility types (typescriptlang.org/docs/handbook); Node.js official docs on the event loop, timers, and `process.nextTick` (nodejs.org/en/docs/guides/event-loop-timers-and-nexttick).
- **Event loop / async deep dives:** multiple write-ups on browser microtask vs. macro-task ordering and Node.js’s libuv phases (timers, pending callbacks, poll, check) plus the

`process.nextTick`/microtask draining rule between phases; CatchJS and Medium write-ups on the `return await` pitfall and unhandled-promise-rejection handling.

- **TypeScript-specific:** DataCamp, CodeForGeek, and GreatFrontEnd's 2026 senior-TypeScript-interview write-ups on utility types (`Partial`, `Pick`, `Omit`, `Record`), discriminated unions, and interfaces-vs-types guidance.
- **Practical coding-question prep:** GreatFrontEnd's front-end interview playbook and machine-coding question sets on implementing `debounce`, `throttle`, `curry`, and deep-clone utilities (the Lodash/Underscore-style functions repeatedly named as popular interview subjects).
- **Debugging scenarios:** OpenReplay and Medium write-ups on diagnosing browser memory leaks with Chrome DevTools heap snapshots, and a documented real-world case of per-request event-listener leaks in Node.js middleware.

A note on freshness

Search results were treated as directional (what topics and phrasings interviewers actually use) rather than as the literal source of every fact in this guide — language mechanics (event loop ordering, closure semantics, TypeScript's structural type system) were cross-checked against MDN, the TypeScript Handbook, and Node.js's own docs rather than taken verbatim from any single blog post.

2 Interview Question Index

ID	Difficulty	Category	Section	Page
Q1	Basic	Core Fundamentals	Core Language Fundamentals	1
Q2	Basic	Core Fundamentals	Core Language Fundamentals	1
Q3	Intermediate	Core Fundamentals	Core Language Fundamentals	2
Q4	Advanced	Core Fundamentals	Core Language Fundamentals	2
Q5	Basic	Closures & Scope	Closures & Scope	4
Q6	Intermediate	Closures & Scope	Closures & Scope	4
Q7	Advanced	Closures & Scope	Closures & Scope	5
Q8	Basic	this, call/apply/bind & Prototypes	this, call/apply/bind & Prototypes	6
Q9	Intermediate	this, call/apply/bind & Prototypes	this, call/apply/bind & Prototypes	6
Q10	Intermediate	this, call/apply/bind & Prototypes	this, call/apply/bind & Prototypes	7
Q11	Advanced	this, call/apply/bind & Prototypes	this, call/apply/bind & Prototypes	7
Q12	Basic	Event Loop (Browser)	The Event Loop (Browser)	9
Q13	Intermediate	Event Loop (Browser)	The Event Loop (Browser)	9
Q14	Advanced	Event Loop (Browser)	The Event Loop (Browser)	10
Q15	Advanced	Event Loop (Browser)	The Event Loop (Browser)	10
Q16	Intermediate	Node.js Event Loop	Node.js Event Loop Specifics	12
Q17	Advanced	Node.js Event Loop	Node.js Event Loop Specifics	12
Q18	Advanced	Node.js Event Loop	Node.js Event Loop Specifics	13
Q19	Basic	Promises & Async/Await	Promises & Async/Await	14
Q20	Intermediate	Promises & Async/Await	Promises & Async/Await	14
Q21	Intermediate	Promises & Async/Await	Promises & Async/Await	15
Q22	Advanced	Promises & Async/Await	Promises & Async/Await	15
Q23	Intermediate	Modules: CommonJS vs. ESM	Modules: CommonJS vs. ES Modules	17
Q24	Advanced	Modules: CommonJS vs. ESM	Modules: CommonJS vs. ES Modules	17
Q25	Intermediate	Practical Coding	Practical Coding Questions	19
Q26	Intermediate	Practical Coding	Practical Coding Questions	19
Q27	Advanced	Practical Coding	Practical Coding Questions	20
Q28	Advanced	Practical Coding	Practical Coding Questions	21
Q29	Basic	TypeScript Fundamentals	TypeScript Fundamentals	22
Q30	Basic	TypeScript Fundamentals	TypeScript Fundamentals	22
Q31	Intermediate	TypeScript Fundamentals	TypeScript Fundamentals	23
Q32	Intermediate	TypeScript Fundamentals	TypeScript Fundamentals	24
Q33	Advanced	TypeScript Advanced	TypeScript Advanced	25

ID	Difficulty	Category	Section	Page
Q34	Advanced	TypeScript Advanced	TypeScript Advanced	25
Q35	Advanced	TypeScript Advanced	TypeScript Advanced	26
Q36	Advanced	TypeScript Advanced	TypeScript Advanced	27
—	—	Common Gotchas & Traps	Common Gotchas & Traps	29
D1–D6	Scenario	Debugging Playbook	Debugging Playbook	30
—	—	Cheat Sheet / Rapid Revision	Cheat Sheet / Rapid Revision	33

3 Core Language Fundamentals

Q1. `var` vs. `let` vs. `const` — what actually differs?

Difficulty: Basic **Category:** Core Fundamentals

`var` is **function-scoped** (or global-scoped if declared outside a function) and is **hoisted** with its declaration initialized to `undefined`, so it's accessible (as `undefined`) before the line it's declared on. `let` and `const` are **block-scoped** and are hoisted too, but into a **Temporal Dead Zone (TDZ)** — accessing them before their declaration throws a `ReferenceError` rather than silently returning `undefined`. `const` additionally forbids reassigning the binding (it does *not* make the value itself immutable — a `const` array or object can still be mutated in place).

```
console.log(a); // undefined (hoisted, not yet assigned)
var a = 1;

console.log(b); // ReferenceError: Cannot access 'b' before initialization (TDZ)
let b = 2;

const arr = [1, 2];
arr.push(3);      // fine -- the ARRAY is mutable
arr = [4, 5];     // TypeError -- the BINDING is constant
```

Common Misconceptions / Traps

Saying “`const` means immutable.” It only freezes the *binding* (the variable name can't be reassigned) — the referenced object/array is fully mutable unless you separately call `Object.freeze()`.

Follow-Up Questions & What's Being Tested

Follow-up: “What is the Temporal Dead Zone, precisely?” Tests whether you know it's not “no hoisting” — `let/const` are hoisted to the top of their block, they're just left in an inaccessible state until their declaration line actually executes.

Q2. `==` vs. `===` — what's the actual rule, and what are the well-known landmine cases?

Difficulty: Basic **Category:** Core Fundamentals

`===` (strict equality) compares both **type** and **value** with no conversion — if the operand types differ, it's immediately `false`. `==` (loose equality) performs **type coercion** first: if the operands are different types, JavaScript converts one or both to a common type (following a specific, quirky algorithm) before comparing.

```
2 == '2'           // true  -- '2' coerced to number 2
2 === '2'          // false -- different types, no coercion
null == undefined  // true  -- special-cased by the spec
null === undefined // false
null == 0          // false -- null only loosely equals undefined (and itself)
NaN == NaN         // false -- NaN is never equal to anything, even itself
[] == false        // true  -- [] -> "" -> 0, false -> 0
```

Common Misconceptions / Traps

Reflexively saying “always use ===, never ==” without being able to explain *why* — that’s the beginner-level answer. A strong answer names the **coercion algorithm** as the actual risk (unpredictable, order-dependent conversions), not == as inherently “wrong.”

Follow-Up Questions & What’s Being Tested

Follow-up: “How do you correctly check if a value is NaN?” Tests whether you know `Number.isNaN(x)` is required — `x === NaN` always returns **false**, even when `x` genuinely holds NaN, because of the IEEE-754 rule that NaN never equals anything.

Q3. Predict the output.

Difficulty: Intermediate **Category:** Core Fundamentals

```
console.log(1 + '1');    // ?
console.log('5' - 1);    // ?
console.log('5' + 1);    // ?
console.log([] + []);    // ?
console.log([] + {});    // ?
console.log({} + []);    // ?
```

Answers: '11' (string on the right forces string concatenation) · 4 (- has no string meaning, so both sides coerce to numbers) · '51' (same rule as line 1) · '' (both arrays coerce to empty strings via `toString()`, then concatenate) · '[object Object]' (the empty object literal `{}` coerces to that string, `[]` to '') · 0 **in a script context**, but '[object Object]' **if typed directly in a console** — because at the start of a statement, an empty `{}` is parsed as an empty **block**, not an object literal, leaving `+[]` as a standalone expression that unary-coerces to 0.

Common Misconceptions / Traps

Missing the `{}` + `[]` case entirely — it’s a genuinely famous gotcha specifically because the *same characters* parse differently depending on statement position (block vs. expression), not because of anything unusual about coercion itself.

Follow-Up Questions & What’s Being Tested

Follow-up: “What’s the general rule the + operator follows?” Tests whether you can state it generally: both operands are converted to primitives; if either resulting primitive is a string, the result is string concatenation, otherwise both sides convert to numbers and it’s numeric addition.

Q4. What is hoisting, precisely, for var declarations, function declarations, and function expressions — and how do they differ?

Difficulty: Advanced **Category:** Core Fundamentals

Hoisting is the JS engine’s compile-phase behavior of registering declarations before executing code line-by-line. **var**: the declaration is hoisted and initialized to **undefined**; the assignment stays in place. **Function declarations** (`function foo() {}`): the *entire function* is hoisted, fully usable before its textual position. **Function expressions** (`const foo = function() {}` or `const foo = () => {}`): only the *variable* declaration hoists (per `var/let/const` rules

above) — the function *value* is not assigned until that line executes, so calling it earlier either gives `undefined` or is not a function (`var`) or a TDZ `ReferenceError` (`let/const`).

```
sayHi(); // "Hi!" -- works, function DECLARATIONS fully hoist
function sayHi() { console.log("Hi!"); }

sayBye(); // TypeError: sayBye is not a function
var sayBye = function() { console.log("Bye!"); };
```

Common Misconceptions / Traps

Treating “hoisting” as one uniform behavior. It’s genuinely three different behaviors (`var`/TDZ/full-function) depending on the declaration form, and conflating them is where most wrong answers come from.

Follow-Up Questions & What’s Being Tested

Follow-up: “Does class declaration hoist the same way as function declaration?” Tests precision — classes hoist like `let/const` (into a TDZ), *not* like function declarations, despite the superficial “it’s a definition at the top of the file” similarity.

4 Closures & Scope

Q5. What is a closure, precisely?

Difficulty: Basic **Category:** Closures & Scope

A **closure** is the combination of a function bundled together with references to its surrounding **lexical scope** — meaning the function retains access to variables from the scope it was *defined* in, even after that outer scope has finished executing and would otherwise have been garbage-collected. Every function in JavaScript forms a closure over its enclosing scope; the term becomes interesting specifically when a function is used *outside* the scope it was created in (returned from another function, passed as a callback, stored for later).

```
function makeCounter() {
  let count = 0;           // "private" -- only reachable via closure
  return function() { return ++count; };
}
const counter = makeCounter();
counter();                 // 1
counter();                 // 2 -- count persisted between calls, via the closure
```

Common Misconceptions / Traps

Defining a closure as “a function inside a function.” That’s neither necessary nor sufficient — the defining property is retained access to outer-scope variables after that outer scope would otherwise be gone, not textual nesting.

Follow-Up Questions & What’s Being Tested

Follow-up: “How would you use a closure to implement private state, and what’s the trade-off vs. a class with a private field?” Tests whether you can connect the mechanism to a real design use (encapsulation via closure, as in `makeCounter` above) and name the trade-off (no shared prototype methods — each closure instance carries its own copy of the inner functions, costing more memory per instance than prototype methods would).

Q6. Predict the output, then fix it.

Difficulty: Intermediate **Category:** Closures & Scope

```
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0);
}
// what prints?
```

Output: 3, 3, 3 — not 0, 1, 2 as many expect. `var` is function-scoped, so there is only **one** `i` shared by all three closures; by the time the `setTimeout` callbacks actually run (after the synchronous loop has already finished), `i` has reached 3. **Fix 1:** change `var` to `let` — `let` creates a **new binding of `i` per loop iteration**, so each closure captures its own separate variable. **Fix 2** (pre-`let` idiom): wrap the body in an IIFE that takes `i` as a parameter, creating a new scope per iteration manually.

```
for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0); // 0, 1, 2 -- each iteration gets its own `i`
}
```

```
}
```

Common Misconceptions / Traps

Explaining this by saying “`let` is block-scoped and `var` isn’t” *without* the second half: that block-scoping specifically means `let` creates a fresh binding *per loop iteration*. Block-scoping alone doesn’t automatically imply per-iteration bindings — that’s a specific, deliberate behavior of `for` loops with `let` in the spec.

Follow-Up Questions & What’s Being Tested

Follow-up: “Does the same per-iteration binding behavior apply to a `while` loop with `let`?” Tests precision — no, the per-iteration fresh-binding rule is specific to the `for` loop’s three-clause syntax; a `while (i < 3) { let i... }` pattern doesn’t have the same mechanism because there’s no loop-clause variable for the spec to special-case.

Q7. How can closures cause a memory leak, and how would you actually find one in production?

Difficulty: Advanced **Category:** Closures & Scope

A closure keeps its entire captured scope alive as long as the closure itself is reachable — if that scope happens to include a large object, a DOM node, or an array, and the closure outlives its usefulness (an event listener that’s never removed, an interval that’s never cleared, a cache with no eviction), everything it references stays in memory too, even though the code “looks” like it should have been freed. A classic real-world case: middleware that attaches a per-request event listener capturing a request ID/logger in its closure, but never removes the listener — after enough requests, thousands of listeners (and everything they capture) accumulate, invisible to normal code review because each individual leak is tiny.

To actually find one: in Chrome DevTools, take a **heap snapshot** at a known-good baseline, exercise the suspected code path repeatedly, take a second snapshot, and use the “Comparison” view to see which constructor’s retained count grew. A rising count of **detached DOM nodes** specifically points at exactly this pattern: a DOM element removed from the page but still referenced (and therefore kept alive) by a closure — typically an event listener on `window/document` that was never cleaned up.

Common Misconceptions / Traps

Assuming closures “cause” memory leaks inherently. They don’t — closures retaining references is the entire point of the mechanism; the leak is specifically an *unbounded lifetime* problem (nothing ever tears the closure down), not a property of closures as a language feature.

Follow-Up Questions & What’s Being Tested

Follow-up: “Give the fix for an interval-based cache that leaks.” Tests whether you reach for the concrete pattern: keep a reference to the interval ID returned by `setInterval`, and expose a `stop()` function that calls `clearInterval` on it — once nothing holds a reference to the closure anymore, it (and whatever it captured) becomes eligible for garbage collection.

5 this, call/apply/bind & Prototypes

Q8. How is this determined in JavaScript?

Difficulty: Basic **Category:** this, call/apply/bind & Prototypes

Unlike lexically-scoped variables, **this** is determined by **how a function is called**, not where it's defined (with one major exception: arrow functions, below). **Plain call** (`fn()`): **this** is **undefined** in strict mode (or the global object in sloppy mode). **Method call** (`obj.fn()`): **this** is `obj`. **Constructor call** (`new Fn()`): **this** is the newly created object. **Explicit binding** (`fn.call(obj)`, `fn.apply(obj)`, `fn.bind(obj)`): **this** is whatever object you pass. **Arrow functions** have *no own this* at all — they capture **this** lexically from their enclosing scope at definition time, and no call-time mechanism can override that.

```
const obj = {
  name: "Ada",
  regular() { return this.name; },
  arrow: () => { return this.name; }, // `this` here is the OUTER scope's this, not
    obj
};
obj.regular(); // "Ada"
obj.arrow();   // undefined (or outer `this.name`) -- arrow ignored obj entirely
```

Common Misconceptions / Traps

Treating **this** as if it worked like a normal lexical variable. It's resolved per call-site for ordinary functions — the exact same function, called two different ways, can have two completely different **this** values.

Follow-Up Questions & What's Being Tested

Follow-up: “Why do React class components historically bind methods in the constructor?” Tests whether you can connect this mechanic to real code you've written — passing `this.handleClick` as a callback (e.g. to `onClick`) detaches it from `this.handleClick()`'s method-call form, so by the time it's invoked, it's effectively a plain call and **this** is **undefined**, unless explicitly bound.

Q9. call, apply, and bind — what's the actual difference?

Difficulty: Intermediate **Category:** this, call/apply/bind & Prototypes

All three let you explicitly set what **this** refers to inside a function call. **call**(`thisArg`, `a`, `b`, `c`): invokes the function *immediately*, with arguments passed *individually*. **apply**(`thisArg`, [`a`, `b`, `c`]): invokes immediately too, but arguments are passed as an *array*. **bind**(`thisArg`, `a`, `b`): does *not* invoke the function — it returns a **new function** with **this** (and optionally some leading arguments) permanently fixed, to be called later.

```
function greet(greeting) { return `${greeting}, ${this.name}`; }
const person = { name: "Grace" };

greet.call(person, "Hi"); // "Hi, Grace" -- called now
greet.apply(person, ["Hi"]); // "Hi, Grace" -- called now, args as array
const bound = greet.bind(person);
bound("Hi"); // "Hi, Grace" -- called LATER, this locked in
```

Common Misconceptions / Traps

Forgetting that `bind` returns a new function without calling it. A common bug: `setTimeout(fn.bind(obj), 1000)` is correct; `setTimeout(fn.bind(obj)(), 1000)` immediately calls `fn` and passes its (likely non-function) *return value* to `setTimeout` instead.

Follow-Up Questions & What's Being Tested

Follow-up: “Once a function is bound, can you re-bind it to something else?” Tests a genuinely obscure but real fact — no, calling `.bind()` again on an already-bound function does not change the originally bound `this`; the first `bind` wins permanently.

Q10. What's the difference between `obj.__proto__` and `Constructor.prototype`?

Difficulty: Intermediate **Category:** `this`, `call`/`apply`/`bind` & Prototypes

`Constructor.prototype` is an ordinary object that lives *on the constructor function itself*, used as the template new instances link to when created with `new`. `obj.__proto__` (a legacy accessor; prefer `Object.getPrototypeOf(obj)`) is a reference *on an instance*, pointing to the object it actually inherits from — for an instance created via `new Constructor()`, `obj.__proto__ === Constructor.prototype`. Property lookups that miss on `obj` itself walk up this chain (`obj` → its `[[Prototype]]` → that object's `[[Prototype]]` → ... → `null`) — the **prototype chain**.

```
function Dog(name) { this.name = name; }
Dog.prototype.bark = function() { return `${this.name} says woof`; };

const rex = new Dog("Rex");
rex.bark(); // "Rex says woof" -- found via the chain
rex.__proto__ === Dog.prototype; // true
rex.hasOwnProperty("bark"); // false -- it's inherited, not own
```

Common Misconceptions / Traps

Believing `instanceof` checks for a *direct* match. It walks the *entire* prototype chain looking for `Constructor.prototype` anywhere in it — `rex instanceof Object` is also `true`, because `Dog.prototype`'s own prototype eventually reaches `Object.prototype`.

Follow-Up Questions & What's Being Tested

Follow-up: “What happens if you mutate `Dog.prototype` after `rex` already exists?” Tests whether you know the chain is a live *reference*, not a copy — `rex` sees the new/changed method immediately, since lookups walk the actual prototype object at call time, not a snapshot taken at construction.

Q11. Why is prototype mutation (“monkey-patching” a built-in prototype) considered dangerous, and what's a real incident this can cause?

Difficulty: Advanced **Category:** `this`, `call`/`apply`/`bind` & Prototypes

Every object sharing that prototype in the *entire* running process — your code, every third-party library, every framework internal — sees the change immediately, because the prototype chain is one shared, live object graph. Two concrete failure modes: (1) **silent breakage** — a library that does `for...in` over an array, or checks `Object.keys()`, can suddenly see

extra enumerable properties you added to `Array.prototype`, breaking assumptions it never documented as fragile; (2) **prototype pollution as a security vulnerability** — if user-controlled input can reach a path like `obj["__proto__"]["polluted"] = true` (a known class of bug in naive deep-merge/object-assignment utilities), an attacker can inject properties onto `Object.prototype` itself, affecting *every plain object in the application*, including ones used for auth checks or config, which has been the root cause of real, disclosed CVEs in popular npm packages.

Common Misconceptions / Traps

Treating this as purely a “style” or “best practice” concern. Prototype pollution is a documented, exploitable **security** vulnerability class (with real CVEs against real, widely-used packages), not just code-cleanliness advice.

Follow-Up Questions & What’s Being Tested

Follow-up: “How would you defend against prototype pollution in a function that merges user-supplied objects into config?” Tests whether you know concrete mitigations: explicitly reject/skip keys named `__proto__`, `constructor`, and `prototype` during merge, use `Object.create(null)` for objects that don’t need a prototype at all, or use `Map` instead of a plain object for untrusted key/value data.

6 The Event Loop (Browser)

Q12. Explain the event loop: call stack, task queue, and how JavaScript's single-threaded model handles async work.

Difficulty: Basic **Category:** Event Loop (Browser)

JavaScript runs on a **single thread** with one **call stack**. Synchronous code executes directly on the stack. Asynchronous work (timers, network responses, user events) doesn't block that thread — instead, the browser's own APIs handle the waiting externally, and when the async work completes, its callback is placed on a **queue** rather than run immediately. The **event loop** is the mechanism that continuously checks: “is the call stack empty?” — and if so, pulls the next callback off a queue and pushes it onto the stack to run. This is why a `setTimeout(fn, 0)` doesn't run instantly: it still has to wait for the current synchronous code *and* anything already queued ahead of it.

Common Misconceptions / Traps

Saying JavaScript is “single-threaded so it can't do anything concurrently.” The *language execution* is single-threaded, but the *runtime* (browser or Node) is not — timers, network I/O, and file I/O happen outside that one JS thread, in the runtime's own infrastructure, which is precisely what makes non-blocking async possible at all.

Follow-Up Questions & What's Being Tested

Follow-up: “What would happen if you ran a synchronous `while(true)` loop?” Tests whether you understand the consequence directly — it would occupy the single call stack forever, so the event loop never gets a chance to check the queue, freezing all timers, network callbacks, and (in a browser) all rendering and user input indefinitely.

Q13. Microtasks vs. macrotasks — what's the actual priority rule, and what goes in each?

Difficulty: Intermediate **Category:** Event Loop (Browser)

Microtasks: Promise callbacks (`.then/.catch/.finally`), `async/await` continuations, and `queueMicrotask()`. **Macrotasks** (also called just “tasks”): `setTimeout/setInterval` callbacks, UI event callbacks, and I/O. The rule: after each single macrotask finishes, the event loop **fully drains the entire microtask queue** — including any *new* microtasks scheduled by microtasks that just ran — *before* it's allowed to run the next macrotask or let the browser repaint.

```
console.log("A");
setTimeout(() => console.log("B"), 0);
Promise.resolve().then(() => console.log("C"));
console.log("D");
// Output: A, D, C, B
// A, D: synchronous code runs first, in order
// C: the microtask queue is drained next, before any macrotask
// B: the setTimeout macrotask runs last, even with a 0ms delay
```

Common Misconceptions / Traps

Assuming `setTimeout(fn, 0)` means “run next.” It means “run as a macrotask, once the stack is clear and *all* pending microtasks have drained” — a chain of `.then()` calls, or code that keeps scheduling new microtasks, can delay that “0ms” timeout arbitrarily.

Follow-Up Questions & What’s Being Tested

Follow-up: “What’s a real performance risk of scheduling too many microtasks?” Tests production awareness — because microtasks must *fully* drain before the browser can repaint or handle the next input event, a runaway chain of microtasks (e.g. a recursive `.then()` that keeps re-scheduling itself) can freeze rendering and make the page feel unresponsive, even though “nothing” is technically blocking synchronously for long.

Q14. Predict the output.

Difficulty: Advanced **Category:** Event Loop (Browser)

```
console.log(1);
setTimeout(() => console.log(2), 0);
Promise.resolve().then(() => {
  console.log(3);
  Promise.resolve().then(() => console.log(4));
});
Promise.resolve().then(() => console.log(5));
console.log(6);
```

Output: 1, 6, 3, 5, 4, 2. Synchronous first: 1, then 6. Then the microtask queue drains: the first `.then` logs 3 and, while running, *schedules a new microtask* (logging 4); the second, independently-queued `.then` logs 5 — but since it was already in the queue *before* the newly-added “log 4” microtask, 5 runs before 4. Only once the microtask queue is completely empty does the macrotask (`setTimeout`, logging 2) finally run.

Common Misconceptions / Traps

Assuming a microtask scheduled *from inside* another microtask jumps the queue. It doesn’t — it goes to the *back* of the microtask queue, behind any microtasks that were already waiting, which is exactly why 5 prints before 4 above despite 4’s `.then` being textually nested inside the one that logs 3.

Follow-Up Questions & What’s Being Tested

Follow-up: “How would the output change if the `setTimeout` had a 100ms delay instead of 0?” Tests whether you know the relative ordering of 1, 6, 3, 5, 4 wouldn’t change at all — all microtasks always finish before any macrotask fires, regardless of the macrotask’s specific delay, since the delay only affects *when* it becomes eligible to run, not its priority relative to microtasks.

Q15. How does `async/await` map onto this model — specifically, at what point does an `async` function actually yield control?

Difficulty: Advanced **Category:** Event Loop (Browser)

An **async** function runs **synchronously** up until its **first await** — everything before that first **await** executes immediately, on the current stack, exactly like ordinary code. At the **await**, the function **suspends** and control returns to the caller; the remainder of the **async** function is scheduled to resume as a **microtask** once the awaited promise settles (this is precisely equivalent to attaching a `.then()` at that point). This is why calling an **async** function doesn't "block" the caller: the caller gets control back at the first **await**, not at the function's end.

```
async function demo() {  
  console.log("1");           // runs synchronously, immediately  
  await null;                 // suspends HERE; rest resumes as a microtask  
  console.log("2");  
}  
console.log("0");  
demo();  
console.log("3");  
// Output: 0, 1, 3, 2
```

Common Misconceptions / Traps

Believing **async** functions run “in the background” from the very first line. The code before the first **await** is fully synchronous and blocks the caller exactly like a normal function call would — the **async** behavior only begins at the suspension point.

Follow-Up Questions & What's Being Tested

Follow-up: “Is `await Promise.resolve(x)` exactly the same speed as not awaiting a already-resolved value?” Tests a subtle, genuinely-debated nuance — no, even an already-settled promise still forces at least one microtask-queue hop before resumption, so **awaiting** a synchronously-available value is not literally free; it still yields to the microtask queue once.

7 Node.js Event Loop Specifics

Q16. What are the phases of Node’s event loop, and how does this differ from the simpler browser model?

Difficulty: Intermediate **Category:** Node.js Event Loop

Node’s event loop (implemented by **libuv**) runs through an ordered sequence of **phases** each iteration, not just one undifferentiated “macrotask queue”: **timers** (due `setTimeout/setInterval` callbacks), **pending callbacks** (certain deferred system-level callbacks), **idle/prepare** (internal use), **poll** (retrieve new I/O events; execute I/O callbacks — this is where most work happens), **check** (`setImmediate` callbacks specifically), and **close callbacks**. The browser’s model, by contrast, doesn’t expose this phase structure — it’s usually taught as a single task queue plus a microtask queue.

Common Misconceptions / Traps

Assuming Node’s event loop is “the same as the browser’s, just with more APIs.” The phased libuv structure is a genuinely different scheduling model underneath — callback ordering questions that seem intuitive from browser experience (like `setTimeout(fn, 0)` vs. `setImmediate(fn)`) can have surprising, *context-dependent* answers in Node specifically because of these phases.

Follow-Up Questions & What’s Being Tested

Follow-up: “`setTimeout(fn, 0)` vs. `setImmediate(fn)` — which runs first?” Tests whether you know the honest answer is “it depends”: at the top level of a script, the order is technically unspecified (timer-phase overhead can go either way); but *inside an I/O callback* (during the poll phase), `setImmediate` always fires first, because the check phase immediately follows poll, while timers would require waiting for a full new loop iteration.

Q17. Where does `process.nextTick()` fit relative to Promise microtasks and the event loop phases?

Difficulty: Advanced **Category:** Node.js Event Loop

`process.nextTick()` schedules a callback onto a queue that is **even higher priority than the Promise microtask queue** in Node — it’s drained *first*, completely, before Node touches the Promise/queueMicrotask microtask queue, and this draining happens **between every single phase transition** of the event loop, not just once per full iteration. Practical ordering: `process.nextTick` queue → Promise microtask queue → next event loop phase.

```
console.log("start");
process.nextTick(() => console.log("nextTick"));
Promise.resolve().then(() => console.log("promise"));
console.log("end");
// Output: start, end, nextTick, promise
```

Common Misconceptions / Traps

Treating `process.nextTick` as “just another microtask, same priority as Promises.” It is a *separate*, strictly higher-priority queue in Node’s implementation — conflating the two is a common source of wrong predictions on Node-specific ordering questions.

Follow-Up Questions & What's Being Tested

Follow-up: “What’s the danger of calling `process.nextTick` recursively/unboundedly?” Tests awareness of **I/O starvation**: because Node fully drains the `nextTick` queue before ever proceeding to the next phase (including the poll phase, where actual I/O callbacks fire), a function that keeps re-scheduling itself via `process.nextTick` can prevent the event loop from ever reaching I/O or timers at all.

Q18. Your Node.js server’s health-check endpoint starts responding slowly, but CPU usage looks low. What would you suspect, and how would you confirm it?

Difficulty: Advanced **Category:** Node.js Event Loop

Low CPU with slow responses is the classic signature of the event loop being **occupied but not computing** — most likely a **blocking synchronous operation** stalling the single JS thread (a large synchronous `JSON.parse/JSON.stringify`, a synchronous file-system call like `fs.readFileSync` on a hot path, a poorly-chosen regular expression suffering **catastrophic backtracking**, or simply an accidental long-running synchronous loop) blocking the event loop from ever reaching the phase where your health-check’s response actually gets sent, even though the CPU itself may be mostly idle while blocked on something like disk I/O. **To confirm:** measure **event loop lag** directly (Node’s `perf_hooks.monitorEventLoopDelay`, or a library like `toobusy-js`, sampling how long a scheduled callback takes to actually fire versus when it was scheduled); if lag spikes correlate with the slow responses, take a CPU profile (`--prof` or the built-in inspector) during a spike to find exactly which synchronous call is hogging the thread.

Common Misconceptions / Traps

Reaching straight for “scale horizontally, add more instances” without first confirming it’s actually a blocking-the-loop problem. Adding instances doesn’t help if every instance independently blocks on the same synchronous bottleneck per request — diagnosing *which* synchronous call is the actual culprit is the higher-value first step.

Follow-Up Questions & What's Being Tested

Follow-up: “How would you fix a genuinely CPU-heavy synchronous computation (not just a bad blocking call) without blocking the event loop?” Tests whether you know the real options: move it to a **worker thread** (`worker_threads`), a separate **child process**, or break it into chunks yielded back to the event loop between chunks (e.g. via `setImmediate`) — `async/await` alone does *not* help here, since it doesn’t move CPU-bound work off the single JS thread.

8 Promises & Async/Await

Q19. What are the three states of a Promise, and why can a Promise only ever “settle” once?

Difficulty: Basic **Category:** Promises & Async/Await

A Promise is **pending** (initial state, neither succeeded nor failed yet), **fulfilled** (completed successfully, holding a resolved value), or **rejected** (failed, holding a reason/error). Once a Promise transitions to fulfilled or rejected, it is **settled** and its state is permanently locked — calling its internal resolve or reject function again afterward has no effect. This one-way, settle-once guarantee is what makes Promises reliably composable: any `.then()` attached to an already-settled Promise is guaranteed to fire (as a microtask) with that same, fixed outcome, no matter when it’s attached.

Common Misconceptions / Traps

Assuming a Promise can be “re-resolved” or reset back to pending. It cannot — if code appears to call `resolve()` twice, the second call is simply a no-op; this is a deliberate, load-bearing guarantee of the spec, not an oversight.

Follow-Up Questions & What’s Being Tested

Follow-up: “What happens if you attach `.then()` to a Promise that already settled five minutes ago?” Tests whether you know settled-Promise callbacks still fire (as a microtask, asynchronously) with the already-determined value/reason — a late `.then()` is not too late.

Q20. Find the bug.

Difficulty: Intermediate **Category:** Promises & Async/Await

```
async function getUser(id) {
  try {
    return fetchUser(id); // fetchUser returns a Promise that can reject
  } catch (err) {
    console.error("failed:", err);
    return null;
  }
}
```

Bug: missing `await` before `fetchUser(id)`. `return fetchUser(id);` exits the `try` block *immediately* with the (still-pending) promise itself — it does not wait for that promise to settle inside this function, so if `fetchUser` later rejects, that rejection happens *after* this function has already returned, completely bypassing the `catch` block. The caller receives a promise that will itself reject, unhandled by *this* function’s error handling at all. **Fix:** `return await fetchUser(id);` — `await` forces the rejection to surface *inside* this function, while the `try` block is still active, where `catch` can actually intercept it.

Common Misconceptions / Traps

This is one of the single most common real-world async bugs: forgetting `return await` inside a `try/catch`, and assuming the surrounding `try` protects a returned promise the same way it protects a returned plain value. It does not — only an *awaited* rejection is catchable by an enclosing `catch`.

Follow-Up Questions & What's Being Tested

Follow-up: “Is `return await` always necessary, even outside a `try/catch`?” Tests nuance — outside a `try/catch`, `return await x` and `return x` behave equivalently to the *caller* (both eventually resolve/reject the same way), so the extra `await` is specifically needed *only* when a `try/catch` needs to observe the rejection itself.

Q21. `Promise.all` vs. `Promise.allSettled` vs. `Promise.race` vs. `Promise.any` — when does each matter?

Difficulty: Intermediate **Category:** Promises & Async/Await

`Promise.all`: resolves when *every* promise resolves, with an array of their values in order — but rejects *immediately* on the *first* rejection, discarding the outcomes of the still-pending ones. **`Promise.allSettled`:** always resolves (never rejects) once every promise has *settled* either way, giving you an array of `{status, value}` or `{status, reason}` objects — use this when you need every result regardless of individual failures (e.g. firing off several independent API calls where one failing shouldn't discard the others' successes). **`Promise.race`:** settles as soon as the *first* promise settles, whether that's a fulfillment or a rejection — useful for timeouts (race a real request against a timer promise). **`Promise.any`:** resolves as soon as the *first* promise *fulfills*, ignoring rejections unless *all* of them reject (in which case it rejects with an `AggregateError`) — useful for “try several redundant sources, take whichever succeeds first.”

Common Misconceptions / Traps

Defaulting to `Promise.all` for a batch of independent requests where a single failure shouldn't sink the whole batch. That's precisely the case `Promise.allSettled` exists for — reaching for *all* there means one failing request silently discards every other result you already had.

Follow-Up Questions & What's Being Tested

Follow-up: “How would you implement a `fetch-with-timeout` using these primitives?” Tests applied knowledge — `Promise.race([fetch(url), new Promise( (_, reject) => setTimeout(() => reject(new Error("timeout")), ms))])`, since `race` settles on whichever of the two finishes first.

Q22. What happens to an unhandled promise rejection, and how do you catch it globally?

Difficulty: Advanced **Category:** Promises & Async/Await

If a Promise rejects and *no* `.catch()` (or awaiting `try/catch`) is attached to handle it by the time the microtask queue finishes processing, the runtime fires an **unhandled rejection** event. In the **browser**, this fires `window.onunhandledrejection` (or the `unhandledrejection` event) and typically logs a console warning, but does *not* crash the page. In **Node.js**, it fires `process.on('unhandledRejection', ...)`, and — since Node 15 — an unhandled rejection with **no listener registered for that event terminates the process by default**, the same as an uncaught synchronous exception. This makes a global handler genuinely important in Node: without one, a single missed `.catch()` somewhere in the codebase can take the whole server down.

```
// Node.js:
```

```
process.on('unhandledRejection', (reason, promise) => {  
  console.error('Unhandled rejection at:', promise, 'reason:', reason);  
  // log/alert here -- but treat this as a safety net, not a substitute  
  // for actually handling errors where they occur  
});
```

Common Misconceptions / Traps

Treating the global handler as the *fix* rather than a *safety net*. A strong answer explicitly says the goal is to *not* rely on it — every promise chain should have its own `.catch()` or enclosing `try/catch`; the global handler exists to catch what genuinely slips through, and to log/alert so the gap gets fixed.

Follow-Up Questions & What's Being Tested

Follow-up: “Why did Node change the default behavior for unhandled rejections in v15?”
Tests awareness of the reasoning — an unhandled rejection almost always indicates a real bug (a forgotten `.catch()`, a typo'd promise chain), and silently swallowing it (the pre-v15 default) let genuine errors go unnoticed in production for a long time; crashing loudly surfaces the bug immediately instead.

9 Modules: CommonJS vs. ES Modules

Q23. CommonJS (`require/module.exports`) vs. ES Modules (`import/export`) — what actually differs, mechanically?

Difficulty: Intermediate **Category:** Modules: CommonJS vs. ESM

CommonJS resolves and loads modules **synchronously**, at the exact point `require()` is called, and the module graph is discovered *dynamically* as code executes — `require()` can be called conditionally, inside an `if`, in a loop, anywhere. **ES Modules** are **statically analyzable**: `import/export` declarations must appear at the top level (not inside conditionals), which lets the engine build the entire dependency graph *before* running any module code, enabling **tree-shaking** (dead-export elimination) and true **live bindings** (an imported value automatically reflects updates in the exporting module, rather than being a one-time copy). Node.js determines which system a file uses via its extension (`.cjs` vs. `.mjs`) or the `"type"` field in the nearest `package.json`.

```
// CommonJS -- exports a snapshot of the value at require-time
// counter.js
let count = 0;
module.exports = { count, increment: () => count++ };

// ESM -- exports a LIVE BINDING
// counter.mjs
export let count = 0;
export function increment() { count++; }
```

Common Misconceptions / Traps

Saying “ESM is just newer syntax for the same thing CommonJS does.” The **live-binding** behavior is a genuine semantic difference, not just syntax — a CommonJS `require` captures the exported object’s shape at require-time (mutating object *properties* is still visible, but reassigning an exported primitive elsewhere in the exporting file is not reflected), while an ESM `import` always sees the exporting module’s current, live value.

Follow-Up Questions & What’s Being Tested

Follow-up: “Can you `require()` an ES Module, or `import` a CommonJS module?” Tests current, practical knowledge — CommonJS `require()` cannot synchronously load an ESM file (ESM’s top-level `await` and async loading make this fundamentally impossible to do synchronously); ESM `import`, on the other hand, *can* import a CommonJS module, typically receiving its `module.exports` object as the default export.

Q24. What is a circular dependency, and how do CommonJS and ESM handle it differently?

Difficulty: Advanced **Category:** Modules: CommonJS vs. ESM

A **circular dependency** is when module A requires/imports module B, and B (directly or transitively) requires/imports A back. **CommonJS**: when the cycle is hit, Node returns whatever partial `module.exports` the in-progress module has built *so far* (since the first `require` call for A hasn’t finished executing A’s file yet) — so the second module often receives an **incomplete** object, commonly missing exports defined *after* the circular `require` line, which is a well-known

source of “why is this import **undefined**” bugs. **ESM**: because bindings are live references resolved against the module graph (not a copy taken at import time), a circularly-imported **function** generally still works correctly even before its module has finished initializing, *as long as it’s not called until both modules have finished loading* — but a circularly-imported *value* accessed too early can still legitimately be **undefined** (or throw a TDZ error for a **let/const** export) until its module finishes running.

Common Misconceptions / Traps

Assuming ESM “fully solves” circular dependencies. It handles function references more gracefully than CommonJS due to live bindings and hoisted function declarations, but circular *value* dependencies can still break in both systems — the real fix in either system is usually to restructure the dependency (extract the shared piece into a third module both sides import), not to rely on either module system’s specific leniency.

Follow-Up Questions & What’s Being Tested

Follow-up: “How would you actually detect a circular dependency in a real codebase?” Tests practical tooling awareness — static analysis tools like **madge** or ESLint’s **import/no-cycle** rule can walk the dependency graph and flag cycles at build/lint time, well before they manifest as a confusing runtime **undefined**.

10 Practical Coding Questions

Q25. Implement debounce.

Difficulty: Intermediate **Category:** Practical Coding

Debounce delays invoking a function until a specified time has passed *without* it being called again — each new call resets the timer. Used for: search-as-you-type (wait until the user pauses typing before firing a request), window-resize handlers (wait until resizing stops before recalculating layout).

```
function debounce(fn, delay) {
  let timeoutId;
  return function (...args) {
    clearTimeout(timeoutId); // cancel any pending call
    timeoutId = setTimeout(() => fn.apply(this, args), delay);
  };
}
// usage: const onSearch = debounce(fetchResults, 300);
```

Common Misconceptions / Traps

Forgetting to preserve **this** and the arguments via **fn.apply(this, args)** — a naive **setTimeout(fn, delay)** loses both the caller's **this** context and whatever arguments the debounced function was actually called with.

Follow-Up Questions & What's Being Tested

Follow-up: “Add a **leading** option so it can optionally fire on the *first* call instead of the last.” Tests whether you can extend the core pattern rather than just reciting a memorized implementation — requires tracking whether a call is currently “in cooldown” separately from the timeout ID itself.

Q26. Implement throttle.

Difficulty: Intermediate **Category:** Practical Coding

Throttle guarantees a function runs at most once per specified interval, *regardless* of how many times it's called during that interval — unlike **debounce**, it fires *during* continuous activity, not just after it stops. Used for: scroll handlers, mousemove handlers, anywhere you want regular, bounded-frequency updates during continuous interaction rather than one final update.

```
function throttle(fn, interval) {
  let lastCall = 0;
  return function (...args) {
    const now = Date.now();
    if (now - lastCall >= interval) {
      lastCall = now;
      fn.apply(this, args);
    }
  };
}
// usage: window.addEventListener('scroll', throttle(onScroll, 100));
```

Common Misconceptions / Traps

Confusing debounce and throttle in conversation — a very common interview tell. Debounce: “wait until it’s quiet, then fire once.” Throttle: “fire at most once every N ms, even while it stays busy.” Getting the names swapped is one of the fastest ways to signal you’ve memorized code without understanding the distinction.

Follow-Up Questions & What’s Being Tested

Follow-up: “This version drops the very last call if it lands mid-interval — how would you make sure the final call still eventually fires (‘trailing’ behavior)?” Tests whether you can reason about a real limitation of the basic implementation, not just reproduce it verbatim.

Q27. Implement a deep clone that handles nested objects and arrays (and discuss what `structuredClone` gives you for free).

Difficulty: Advanced **Category:** Practical Coding

```
function deepClone(value, seen = new WeakMap()) {
  if (value === null || typeof value !== "object") return value; // primitives:
  // copy by value already
  if (seen.has(value)) return seen.get(value); // handle
  // circular references

  const clone = Array.isArray(value) ? [] : {};
  seen.set(value, clone);
  for (const key of Object.keys(value)) {
    clone[key] = deepClone(value[key], seen);
  }
  return clone;
}
```

The modern, built-in answer for most cases is `structuredClone(value)` (available in browsers and Node 17+), which correctly handles circular references, `Date`, `Map`, `Set`, `RegExp`, and typed arrays natively — things a naive hand-rolled `JSON.parse(JSON.stringify(x))` clone silently gets wrong (it drops `undefined` values and functions, converts `Date` objects to strings, and throws on circular references entirely).

Common Misconceptions / Traps

Reaching for `JSON.parse(JSON.stringify(x))` as a “good enough” deep clone without naming its failure modes. It’s a reasonable quick hack for plain, JSON-safe data, but silently mangles `Date`/`Map`/`Set`/`undefined`/functions and outright throws on circular references — an interviewer asking this question is usually specifically listening for you to know that.

Follow-Up Questions & What’s Being Tested

Follow-up: “Why does the `WeakMap` matter here specifically, as opposed to a regular `Map`?” Tests whether you understand it’s about not *preventing* garbage collection of the original objects being cloned — a `WeakMap`’s keys don’t count as strong references, so this tracking structure doesn’t itself keep the source objects alive after cloning finishes.

Q28. Implement curry for a function of arbitrary arity.

Difficulty: Advanced **Category:** Practical Coding

Currying transforms a function that takes multiple arguments into a sequence of functions that each take one (or a subset of) argument(s), returning a new function until all arguments have been supplied, at which point it invokes the original.

```
function curry(fn) {
  return function curried(...args) {
    if (args.length >= fn.length) {           // fn.length = number of declared
      params
      return fn.apply(this, args);
    }
    return (...more) => curried.apply(this, [...args, ...more]);
  };
}

const add3 = (a, b, c) => a + b + c;
const curried = curry(add3);
curried(1)(2)(3);           // 6
curried(1, 2)(3);           // 6 -- also works, args can be grouped any way
curried(1, 2, 3);           // 6 -- also works, all at once
```

Common Misconceptions / Traps

Relying on `fn.length` without knowing its limitation: it only counts parameters *before* the first default value or rest parameter, and doesn't count them at all past a rest parameter — `curry` implemented this way silently breaks for functions with default or rest parameters, since `fn.length` under-reports the real arity.

Follow-Up Questions & What's Being Tested

Follow-up: “How would you curry a function whose arity you can't rely on `fn.length` for?”
Tests whether you know the honest answer: you generally can't infer it automatically in that case — the caller has to explicitly pass the intended arity as a second argument to your `curry` function.

11 TypeScript Fundamentals

Q29. What does it mean that TypeScript’s type system is structural, not nominal?

Difficulty: Basic **Category:** TypeScript Fundamentals

TypeScript checks type compatibility by **shape** (what properties/methods exist and their types), not by **declared name or lineage**. Two independently-declared types with identical shapes are considered interchangeable, even though neither “knows about” the other — this is the opposite of nominal typing (as in Java/C#), where two classes with identical members are still incompatible types unless one explicitly extends/implements the other.

```
interface Point { x: number; y: number; }
class Vector { x: number; y: number; constructor(x: number, y: number) { this.x = x;
  this.y = y; } }

function logPoint(p: Point) { console.log(p.x, p.y); }
logPoint(new Vector(1, 2));      // OK -- Vector has the right SHAPE, even though it
  never mentioned Point
logPoint({ x: 1, y: 2 });        // also OK -- a plain object literal with the right
  shape
```

Common Misconceptions / Traps

Assuming TypeScript works like Java, where you must explicitly **implements** an interface to be assignable to it. TypeScript never requires that — any value with a compatible shape satisfies the type, whether or not it declares any relationship to it.

Follow-Up Questions & What’s Being Tested

Follow-up: “Does this mean an object with *extra* properties is always assignable to a narrower type?” Tests a specific, commonly-missed exception: yes for a variable already holding that value, but TypeScript applies **excess property checking** specifically to *object literals* assigned or passed *directly* — `logPoint({x:1, y:2, z:3})` is an error (excess `z`), even though the exact same shape stored in a variable first and then passed would be allowed.

Q30. **interface** vs. **type** — when does the choice actually matter?

Difficulty: Basic **Category:** TypeScript Fundamentals

Both can describe an object’s shape, and for plain object shapes they’re largely interchangeable day-to-day. The real differences: **interface** supports **declaration merging** (declaring the same interface name twice *adds* to it rather than conflicting — useful for extending third-party/ambient types) and is generally preferred for public object/class APIs meant to be extended. **type** is strictly more flexible: it can alias **unions**, **tuples**, **primitives**, and **mapped/conditional types** — things **interface** cannot express at all.

```
type Status = "idle" | "loading" | "success" | "error"; // union -- interface
  CANNOT do this
type Pair = [string, number];                          // tuple -- interface
  cannot do this either

interface User { id: string; name: string; }
```

```
interface User { email: string; } // declaration merging: User now has id,
name, AND email
```

Common Misconceptions / Traps

Treating this as a purely stylistic/team-preference choice with zero technical difference. Declaration merging is a genuine, sometimes load-bearing capability (e.g. augmenting `Express.Request` in Express middleware type definitions relies on it) that only **interface** provides.

Follow-Up Questions & What's Being Tested

Follow-up: “Give a concrete example where only **type** works and **interface** genuinely cannot.” Tests whether the distinction is actually internalized, not just recited — any union type (`Status` above), any tuple, or a mapped type like `{ [K in keyof T]: boolean }` are all things **interface** structurally cannot express.

Q31. What is type narrowing, and what are the common ways to trigger it?

Difficulty: Intermediate **Category:** TypeScript Fundamentals

Narrowing is TypeScript refining a variable’s statically-known type to a more specific subtype within a particular code branch, based on a runtime check the compiler can statically recognize — letting you safely access members that only exist on the narrower type. Common triggers: **typeof** checks, **instanceof** checks, **in** checks (property existence), equality checks against literal values, truthiness checks, and user-defined **type predicate** functions (`function isX(v): v is X`).

```
function process(value: string | number) {
  if (typeof value === "string") {
    value.toUpperCase(); // TS knows value: string here
  } else {
    value.toFixed(2); // TS knows value: number here (the ONLY other branch of
                     // the union)
  }
}

function isFish(pet: Fish | Bird): pet is Fish { // user-defined type predicate
  return (pet as Fish).swim !== undefined;
}
```

Common Misconceptions / Traps

Assuming narrowing works through *any* function call, not just inline checks. Calling an ordinary helper function like `if (checkIsString(value))` does *not* narrow `value` unless that helper is specifically declared with a `value is string` type predicate return type — TypeScript can’t see through an arbitrary function’s internal logic.

Follow-Up Questions & What's Being Tested

Follow-up: “What’s a discriminated union, and how does it make narrowing more reliable than ad-hoc **typeof**/**in** checks?” Tests whether you can connect this to Q33/Section 12’s deeper pattern — a shared literal “tag” property lets the compiler narrow an entire union by

switching on *one* property, exhaustively and safely, rather than checking individual optional fields' presence.

Q32. What is **unknown**, and why is it considered safer than **any**?

Difficulty: Intermediate **Category:** TypeScript Fundamentals

any **opts a value out of type checking entirely** — you can call any method, access any property, assign it to anything, with zero compiler safety net, and that unsafety *propagates*: anything derived from an **any** value is also implicitly **any**. **unknown** also accepts *any* value (both are “top types”), but the compiler **forbids doing anything with it** until you’ve narrowed it to a more specific type first (via **typeof**, **instanceof**, a type predicate, or an explicit assertion) — it’s the type-safe way to say “I don’t know this value’s type yet,” as opposed to **any**’s “stop checking this value’s type.”

```
function handleAny(x: any) { x.toUpperCase(); }           // compiles -- but crashes at
    runtime if x isn't a string
function handleUnknown(x: unknown) {
  // x.toUpperCase();                                     // COMPILE ERROR -- must
  narrow first
  if (typeof x === "string") x.toUpperCase();             // fine, now narrowed to
  string
}
```

Common Misconceptions / Traps

Using **any** as a quick way to “make a type error go away.” It doesn’t fix the underlying uncertainty, it just hides it from the compiler — and because **any** propagates through everything it touches, one **any** can silently disable type checking across a much larger surface area than where it was originally written.

Follow-Up Questions & What’s Being Tested

Follow-up: “Where does data typically enter a codebase as **unknown** in practice?” Tests whether you connect this to a real workflow — the return value of **JSON.parse()**, data from an external API response, or a **catch (err)** clause’s error variable are all places TypeScript reasonably can’t know the real type, and **unknown** (rather than **any**) forces you to validate/narrow before trusting the shape.

12 TypeScript Advanced

Q33. Write a discriminated union modeling an async request's state (idle/loading/success/error), and explain why this is safer than one object with optional fields.

Difficulty: Advanced **Category:** TypeScript Advanced

```
type RequestState<T> =
  | { status: "idle" }
  | { status: "loading" }
  | { status: "success"; data: T }
  | { status: "error"; error: string };

function render<T>(state: RequestState<T>) {
  switch (state.status) {
    case "idle": return "Nothing yet";
    case "loading": return "Loading...";
    case "success": return state.data; // TS knows `data` exists ONLY in this branch
    case "error": return state.error; // TS knows `error` exists ONLY in this branch
  }
}
```

The shared literal `status` field is the **discriminant** — switching/branching on it lets TypeScript narrow the *entire* object to the exact matching variant, so `state.data` is only accessible where `status` is genuinely "success". Compare this to a single flat type like `{ status: string; data?: T; error?: string; loading?: boolean }`: nothing stops you from constructing an **invalid, contradictory state** (`loading: true` *and* `data` both set), and the compiler can't help you remember to check `error` before reading `data`, because both are just independently optional.

Common Misconceptions / Traps

Modeling UI/async state with a bag of optional booleans/fields instead of a discriminated union. It's not just a style preference — it makes genuinely invalid states (`loading` *and* `error` *and* `data` all simultaneously "present") representable in the type system, which is exactly the class of bug discriminated unions exist to make impossible to construct in the first place.

Follow-Up Questions & What's Being Tested

Follow-up: "What happens if you add a new `status` variant but forget to handle it in this `switch`?" Tests whether you know the **exhaustiveness-checking** pattern: assigning the unreachable `state` in a default case to a variable typed `never` (`const _exhaustive: never = state;`) makes the compiler itself flag any unhandled variant as an error the moment a new one is added — turning a silent runtime gap into a compile-time one.

Q34. Implement `Pick` and `Partial` from scratch using mapped types, without using the built-in utility types.

Difficulty: Advanced **Category:** TypeScript Advanced

```
type MyPick<T, K extends keyof T> = {
```

```

[P in K]: T[P];
};

type MyPartial<T> = {
  [P in keyof T]?: T[P];
};

interface User { id: string; name: string; email: string; }
type UserPreview = MyPick<User, "id" | "name">;    // { id: string; name: string }
type UserPatch = MyPartial<User>;                 // { id?: string; name?: string;
  email?: string }

```

[P in K] is a **mapped type**: it iterates over every key in the union K and produces a corresponding property in the result. K **extends** `keyof T` constrains K to actually be a valid subset of T's own keys (the compiler rejects a typo'd key at the call site). `T[P]` (an **indexed access type**) looks up the original property's type for each key being copied over.

Common Misconceptions / Traps

Being able to *use* `Pick`/`Partial` fluently but freezing when asked to implement them — this question specifically tests whether “mapped type” and “indexed access type” are concepts you actually understand, versus utility types you've only ever consumed as opaque black boxes.

Follow-Up Questions & What's Being Tested

Follow-up: “Now implement `MyReadOnly<T>`.” Tests whether you can extend the pattern under pressure: `type MyReadOnly<T> = { readonly [P in keyof T]: T[P] };` — the same mapped-type shape, with the `readonly` modifier added per property.

Q35. What is a conditional type, and what problem do they solve that a plain union can't?

Difficulty: Advanced **Category:** TypeScript Advanced

A **conditional type** (`T extends U ? X : Y`) picks between two types based on a type-level condition, evaluated **per member** when T is a union (this per-member evaluation is called a **distributive conditional type**). This lets you express type-level *transformations* — deriving a new type's shape *from* another type's structure — which a plain, manually-written union cannot do, because a union is just a fixed, enumerated list of possibilities with no ability to react to what gets plugged in.

```

type NonNullableCustom<T> = T extends null | undefined ? never : T;
type Result = NonNullableCustom<string | null | undefined>;    // string
                                                                // (distributes
                                                                // over the union,
                                                                // filters each
                                                                // member individually,
                                                                // `never` members
                                                                // vanish from the result)

// infer: extract a type FROM another type
type UnwrapPromise<T> = T extends Promise<infer U> ? U : T;
type A = UnwrapPromise<Promise<string>>;    // string
type B = UnwrapPromise<number>;             // number -- not a Promise, passes
through unchanged

```

Common Misconceptions / Traps

Not knowing that conditional types **distribute** over unions automatically when the checked type is a “naked” type parameter — this is precisely why **never** members (like the filtered-out **null/undefined** above) simply disappear from the resulting union instead of turning the whole result into **never**, which surprises people expecting ordinary function-like “evaluate once” semantics.

Follow-Up Questions & What’s Being Tested

Follow-up: “What does **infer** actually do in `T extends Promise<infer U> ? U : T?`” Tests whether you can explain the mechanism, not just paste the pattern — **infer U** introduces a new type variable that TypeScript solves for by structurally matching **T** against the pattern `Promise<...>`; if the match succeeds, **U** is bound to whatever filled that slot, usable in the “true” branch.

Q36. TypeScript `enum` vs. a union of string literals (or a `const` object) — what do senior interviewers actually want to hear here?

Difficulty: Advanced **Category:** TypeScript Advanced

A TypeScript **enum** generates **real runtime JavaScript** (an actual object, with — for numeric enums — a confusing **reverse mapping** from value back to name built in), and has type behavior that surprises people: a numeric enum’s members are compatible with plain numbers in ways that can defeat the type safety you’d expect, while string enum members are *not* interchangeable with plain string literals of the same text without an explicit cast. A **union of string literals** (`type Status = "idle" | "loading" | "error"`) or a **const** object with **as const** produces **no extra runtime code at all**, behaves exactly like the plain strings/values you’d expect, and composes cleanly with mapped types, discriminated unions, and template literal types — which is why current guidance increasingly favors literal unions over **enum** for most everyday cases (UI state, API status codes, component variants), reserving actual **enum** for cases that specifically want its runtime object and reverse-mapping behavior.

```
enum Status { Idle, Loading, Error }           // generates a real JS object at runtime
// , with quirky typing
type StatusLiteral = "idle" | "loading" | "error"; // zero runtime cost, plain
// strings, composes cleanly

const Color = { Red: "red", Green: "green" } as const;
type ColorType = typeof Color[keyof typeof Color]; // "red" | "green" -- the
// common const-object pattern
```

Common Misconceptions / Traps

Reflexively reaching for **enum** out of habit from other languages (Java/C#) without being able to articulate the trade-off. In 2026-era TypeScript interviews specifically, this question is less about “do you know enum syntax” and more about whether you understand *why* many teams and style guides have moved toward literal unions/**const** objects instead.

Follow-Up Questions & What's Being Tested

Follow-up: “What is a `const enum`, and why doesn't it fully solve the runtime-cost concern?” Tests depth — `const enum` is inlined at compile time (no runtime object emitted), which sounds like the best of both, but it's incompatible with `isolatedModules` (used by most modern single-file transpilers like esbuild/SWC/Babel, which can't safely inline across files), so many toolchains explicitly disallow it.

13 Common Gotchas & Traps

A dense, rapid-fire list of the specific misconceptions that come up again and again across the sources consulted for this guide — most already appear woven into a question above; this section exists so you can review just the traps, without re-reading every full answer.

`typeof null === "object"`

A long-standing spec bug, kept for backward compatibility. Always check `value === null` explicitly rather than trusting `typeof`.

Array `sort()` is not numeric by default

`[10, 2, 1].sort()` gives `[1, 10, 2]` — it sorts by **string** comparison unless you pass a comparator: `.sort((a, b) => a - b)`.

`Object.freeze` is shallow

Freezing an object only locks its *own, top-level* properties — a nested object inside a frozen object remains fully mutable.

`parseInt("08")` without a radix

Historically could be interpreted as octal in older engines; always pass the radix explicitly: `parseInt(str, 10)`.

Floating-point precision

`0.1 + 0.2 !== 0.3` (it's `0.30000000000000004`) — a consequence of IEEE-754 binary floating point, not a JavaScript-specific bug. Compare with a small epsilon tolerance, or use integer cents/minor-units for currency.

Array holes vs. `undefined`

`[1, , 3]` has a genuine **hole** at index 1, which behaves differently from an explicit `undefined` for methods like `forEach` (which skips holes entirely) vs. `map` (which also skips them but preserves the hole in the output).

`setTimeout` return value differs by environment

In the browser it returns a numeric ID; in Node.js it returns a `Timeout` **object**. Code assuming a number (e.g. for storage/comparison) can break when ported between environments.

TypeScript's `as` is not a runtime check

A type assertion (`value as User`) tells the compiler to *trust you* — it performs zero runtime validation, so an incorrectly-asserted value compiles cleanly and then fails unpredictably later, often far from where the bad assertion was made.

Optional chaining short-circuits the whole chain

`a?.b.c.d` — if `a` is nullish, the entire expression short-circuits to `undefined` immediately; it does *not* continue evaluating `.c.d` on `undefined` and throw.

`Array.prototype.includes` vs. `indexOf` for NaN

`[NaN].indexOf(NaN)` returns `-1` (uses `===` internally), but `[NaN].includes(NaN)` correctly returns `true` (uses the `SameValueZero` algorithm) — a genuine, useful difference beyond “includes reads nicer.”

14 Debugging Playbook

Every scenario below follows the same shape a strong candidate should actually reason through out loud: **symptoms** → **hypotheses** → **checks** → **root cause** → **fix**.

D1 — A dashboard tab's memory grows steadily over a 6-hour session

Symptoms: Browser tab memory climbs from 150MB to over 1GB across a long session; the page eventually becomes sluggish.

Hypotheses: an event listener attached repeatedly (e.g. on every render/poll) but never removed; a growing cache/array with no eviction; detached DOM nodes kept alive by a closure.

Checks: Chrome DevTools **Memory** panel — take a heap snapshot at startup, exercise the suspected flow (e.g. let a polling loop run for a few minutes), take a second snapshot, and use the **Comparison** view sorted by **#Delta** to see which constructor's count grew fastest.

Root cause & fix: A rising **Detached HTMLDivElement** (or similar) count almost always means a listener on **window/document** captured a DOM node in its closure and was never cleaned up with **removeEventListener** — fix by tracking the listener reference and removing it in the corresponding teardown (component unmount, **stop()** function, etc.). If instead a plain array/object/Map is the fastest-growing constructor, look for a cache or log buffer with no size cap or expiry.

D2 — A test suite has intermittent, unreproducible failures

Symptoms: A specific async test fails maybe 1 in 20 runs, always with a slightly different assertion, and passes again on retry with no code change.

Hypotheses: a **race condition** — the test asserts before an async operation it depends on has actually settled; two async operations assumed to run in a specific order that isn't actually guaranteed; a shared/global test fixture leaking state between tests that run out of order.

Checks: run the single failing test in a tight loop (e.g. **--repeat 50**) to raise the failure frequency; add temporary logging with high-resolution timestamps around every **await** in the code path to see the *actual* completion order across runs.

Root cause & fix: Look specifically for a missing **await** (the test moves on before a promise the code under test kicked off has settled), or a **Promise.all** where the test's assertions implicitly assume a specific completion *order* that **Promise.all** never actually guarantees (it preserves input *array* order in its result, not completion order). Fix by awaiting explicitly at every dependency point, and asserting on the resolved values rather than on timing/ordering assumptions.

D3 — “Cannot read properties of undefined” in production, but it works locally

Symptoms: A runtime **TypeError** on a property access that TypeScript compiled without complaint; reproduces only against the production API, not local mock data.

Hypotheses: the real API response's shape doesn't actually match the TypeScript type describing it (the type was hand-written or drifted from a schema change, and TypeScript trusted it with no runtime check); an unsafe **as** assertion somewhere upstream; an optional field the type marks as required, or vice versa.

Checks: log (or breakpoint on) the raw API response *before* any typed access, and compare

its actual shape against the TypeScript interface describing it; grep the data path for **as** assertions between the fetch and the failing access.

Root cause & fix: This is the single most common gap TypeScript *cannot* close on its own — types are a compile-time-only promise about data you don't actually control (an external API). Fix by validating the response at the boundary with a runtime schema validator (e.g. Zod, io-ts) that both checks the shape *and* produces the TypeScript type from the same source of truth, so drift becomes a caught runtime error at the boundary instead of an uncaught one deep in application code.

D4 — The UI freezes for a couple of seconds during a large list render/update

Symptoms: Clicking a button that updates a large list (thousands of rows) makes the entire page unresponsive to input for 1–3 seconds.

Hypotheses: a synchronous **long task** occupying the main thread (a large synchronous computation, sort, or filter running on the full dataset every render); an expensive operation re-run unnecessarily due to a missing memoization boundary.

Checks: Chrome DevTools **Performance** panel — record while triggering the update, look for a long yellow (scripting) block on the main thread flame chart; note which function occupies most of that block's self-time.

Root cause & fix: If it's a genuinely necessary heavy computation, break it into chunks yielded back to the event loop (e.g. processing in batches via `setTimeout/requestIdleCallback` between chunks) so the browser can still paint and respond to input between them, or move it off the main thread entirely with a **Web Worker**. If profiling shows the same expensive computation re-running when its inputs haven't actually changed, the fix is a memoization boundary instead (e.g. recomputing a derived, sorted/filtered list only when its source data or filter criteria actually change).

D5 — A Node.js process exits unexpectedly with no application error logged

Symptoms: The process just stops; application logs show nothing unusual right before the exit; no crash stack trace in the app's own logging.

Hypotheses: an **unhandled promise rejection** with no `process.on('unhandledRejection', ...)` listener registered (Node 15+ terminates the process by default in this case, Q22); an uncaught synchronous exception escaping an event handler.

Checks: check the process manager's own logs (PM2/systemd/container runtime), not just the application's logger — Node prints an “UnhandledPromiseRejection” warning *to stderr* before exiting, which is easy to miss if only stdout/application-level logs are being collected.

Root cause & fix: Register `process.on('unhandledRejection', ...)` and `process.on('uncaughtException', ...)` globally, log the full reason/stack immediately, and treat both as **alerts to fix the root cause**, not as things to keep the process alive through — after logging, it's standard practice to still exit deliberately (the process may be in an inconsistent state), but now with the actual cause captured. Then find and fix the specific missing `.catch()/try` that let it slip through in the first place.

D6 — A TypeScript build passes cleanly, but a value is the wrong type at runtime

Symptoms: No compiler errors anywhere, but a value logged at runtime clearly isn't the type its declared type claims (e.g. a variable typed `User` is actually `undefined`).

Hypotheses: an unsafe `as` type assertion somewhere upstream silencing the compiler about a value the code hasn't actually verified; a generic function's type parameter inferred incorrectly from a loosely-typed call site; `any` having silently entered and propagated through several calls' worth of code.

Checks: `grep` the data's path from its source for `as` and `any`; temporarily add `// @ts-expect-error` above suspected lines to confirm whether the compiler would actually catch a given misuse if the assertion/`any` weren't there.

Root cause & fix: Remember: TypeScript's guarantees are **compile-time only** and are only as strong as the weakest `as/any` anywhere upstream of the value in question — the fix is almost always to replace the unsafe assertion with a real runtime check (a type predicate, a schema validator) at the exact point uncertain data enters the typed portion of the codebase, rather than asserting past the uncertainty and hoping.

15 Cheat Sheet / Rapid Revision

Equality & coercion, compressed

Expression	Result & why
<code>null == undefined</code>	<code>true</code> — special-cased by spec (only equal to each other and themselves)
<code>NaN === NaN</code>	<code>false</code> — use <code>Number.isNaN(x)</code> , never <code>x === NaN</code>
<code>[] == false</code>	<code>true</code> — <code>[]</code> → <code>"</code> → <code>0</code> , <code>false</code> → <code>0</code>
<code>typeof null</code>	<code>"object"</code> — long-standing spec bug, check <code>=== null</code> explicitly
<code>0.1 + 0.2</code>	<code>0.30000000000000004</code> — IEEE-754 binary float, not a JS bug

Event loop priority, compressed

Priority (highest first)	Contains
1. Synchronous code	Everything on the call stack, runs to completion first
2. <code>process.nextTick</code> (Node only)	Node-specific, higher priority than Promise microtasks
3. Microtasks	Promise <code>.then/.catch/.finally</code> , <code>async/await</code> resumption, <code>queueMicrotask</code>
4. Macrotasks	<code>setTimeout/setInterval</code> , I/O callbacks, UI events (browser: one at a time, draining all microtasks between each)

this-binding rules, compressed

Call form	this is
<code>fn()</code>	<code>undefined</code> (strict mode)
<code>obj.fn()</code>	<code>obj</code>
<code>new Fn()</code>	the newly created instance
<code>fn.call/apply/bind(obj)</code>	<code>obj</code> , set explicitly
Arrow function	lexical <code>this</code> from the enclosing scope — ignores all of the above entirely

Promise combinators, compressed

`Promise.all` — all-or-nothing, rejects on first rejection · `Promise.allSettled` — always resolves, gives you every outcome · `Promise.race` — settles on whichever finishes first, success or failure · `Promise.any` — resolves on first *success*, rejects only if all fail.

TypeScript utility types, compressed

Utility	What it does
<code>Partial<T></code>	Makes every property of <code>T</code> optional

Utility	What it does
<code>Required<T></code>	Makes every property of <code>T</code> required (opposite of <code>Partial</code>)
<code>Pick<T, K></code>	New type with only keys <code>K</code> from <code>T</code>
<code>Omit<T, K></code>	New type with every key of <code>T</code> <i>except</i> <code>K</code>
<code>Record<K, V></code>	An object type with keys <code>K</code> , all values of type <code>V</code>
<code>Readonly<T></code>	Makes every property of <code>T</code> <code>readonly</code>
<code>ReturnType<F></code>	Extracts a function type <code>F</code> 's return type

Day-before-the-interview checklist

```
[ ] Can you trace A, D, C, B for the classic sync/promise/setTimeout output question?
[ ] Can you explain why `for (var i...)` + setTimeout prints the final value N times?
[ ] Can you write debounce AND throttle from memory, and state which is which?
[ ] Can you explain why `return fetchX()` (no await) breaks try/catch?
[ ] Can you implement Pick<T,K> and Partial<T> as mapped types from scratch?
[ ] Can you state the actual difference between unknown and any?
[ ] Can you name process.nextTick's priority relative to Promise microtasks?
[ ] Have you redone every "Find the bug" question without looking at the answer first?
```